

# End-to-End DevOps Project on AWS: Runbook

Terraform · EKS · Jenkins CI/CD · GitOps with ArgoCD · EFS · ALB · Trivy · Prometheus · Grafana

This runbook takes you from an empty AWS account to a running, monitored and security-scanned application. Every step shows the exact commands, what you should see, and how to check it worked before you move on.

Repository: <https://github.com/gh-vishwesh/End-to-End-DevOps-Project>

Region: ap-south-1 (Mumbai) · EKS cluster: testing\_k8s · App namespace: k8s-project

Version: 06 October 2026 | Author: Vishwesh Pandey

# Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>0. How the pieces fit together</b>                            | <b>4</b>  |
| 0.1 The flow of a code change                                    | 4         |
| 0.2 What runs where                                              | 4         |
| 0.3 Repository layout                                            | 4         |
| 0.4 Time and cost                                                | 5         |
| <b>1. Prerequisites</b>                                          | <b>5</b>  |
| 1.1 Tools on your workstation (Linux / WSL2 Ubuntu)              | 5         |
| 1.2 AWS account and credentials                                  | 5         |
| 1.3 GitHub repository and token                                  | 6         |
| 1.4 Clone the repo                                               | 6         |
| <b>2. Phase 1: Provision AWS infrastructure with Terraform</b>   | <b>6</b>  |
| Step 1.1: Create the Terraform state bucket (one time only)      | 6         |
| Step 1.2: Set your variables                                     | 6         |
| Step 1.3: Init, plan, apply                                      | 7         |
| Step 1.4: Record the outputs                                     | 7         |
| Step 1.5: Verify in AWS                                          | 7         |
| <b>3. Phase 2: Connect to EKS and install cluster add-ons</b>    | <b>8</b>  |
| Step 2.1: Configure kubectl                                      | 8         |
| Step 2.2: Install the EFS CSI driver                             | 8         |
| Step 2.3: Install the AWS Load Balancer Controller               | 8         |
| Step 2.4: Point the StorageClass at the new EFS file system      | 8         |
| <b>4. Phase 3: Jenkins CI server</b>                             | <b>9</b>  |
| Step 3.1: SSH to the Jenkins server                              | 9         |
| Step 3.2: Install Docker, AWS CLI, Java 21 and Jenkins           | 9         |
| Step 3.3: Unlock Jenkins                                         | 9         |
| Step 3.4: Add the GitHub token as a credential                   | 9         |
| Step 3.5: Create the pipeline job                                | 9         |
| Step 3.6: Add the GitHub webhook                                 | 10        |
| Step 3.7: Run the first build                                    | 10        |
| <b>5. Phase 4: Security scanning (DevSecOps)</b>                 | <b>11</b> |
| <b>6. Phase 5: GitOps with ArgoCD</b>                            | <b>11</b> |
| Step 5.1: Install ArgoCD                                         | 11        |
| Step 5.2: Open the UI and log in                                 | 11        |
| Step 5.3: Create the Application                                 | 11        |
| Step 5.4: Check what ArgoCD deployed                             | 11        |
| Step 5.5: Open the application                                   | 12        |
| <b>7. Phase 6: Prove the full CI/CD loop</b>                     | <b>12</b> |
| <b>8. Phase 7: In-cluster monitoring (kube-prometheus-stack)</b> | <b>13</b> |

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| Step 7.1: Install . . . . .                                                     | 13        |
| Step 7.2: Open Grafana . . . . .                                                | 13        |
| Step 7.3: Open Prometheus and Alertmanager (port-forward) . . . . .             | 13        |
| Step 7.4: Email alerts and Jenkins metrics (optional) . . . . .                 | 13        |
| <b>9. Phase 8: Monitoring server on EC2 (Prometheus, Alertmanager, Grafana)</b> | <b>14</b> |
| Step 8.1: Prometheus . . . . .                                                  | 14        |
| Step 8.2: node_exporter on both servers . . . . .                               | 14        |
| Step 8.3: Pushgateway and Alertmanager . . . . .                                | 14        |
| Step 8.4: Configuration and alert rules . . . . .                               | 14        |
| Step 8.5: Email notifications . . . . .                                         | 14        |
| Step 8.6: Grafana . . . . .                                                     | 14        |
| <b>10. Phase 9: HTTPS with Route 53 and ACM (optional)</b>                      | <b>15</b> |
| <b>11. Verification checklist</b>                                               | <b>15</b> |
| <b>12. Troubleshooting</b>                                                      | <b>16</b> |
| Terraform . . . . .                                                             | 16        |
| Jenkins . . . . .                                                               | 16        |
| Kubernetes / ArgoCD . . . . .                                                   | 16        |
| Useful commands . . . . .                                                       | 17        |
| <b>13. Phase 10: Day-2 operations</b>                                           | <b>17</b> |
| <b>14. Phase 11: Teardown (avoid charges)</b>                                   | <b>17</b> |
| <b>15. Security summary</b>                                                     | <b>18</b> |

## 0. How the pieces fit together

### 0.1 The flow of a code change

1. You push a change to the `main` branch on GitHub.
2. A GitHub webhook calls Jenkins (`http://<jenkins-ip>:8080/github-webhook/`).
3. Jenkins clones the repo, runs the Django unit tests, and runs the Trivy secret, IaC and dependency scans.
4. Jenkins builds the Docker image, scans it with Trivy (any fixable CRITICAL CVE stops the pipeline), and pushes it to Amazon ECR.
5. Jenkins writes the new image tag into `Kubernetes with ArgoCD/appdeployment.yml` and pushes that commit back to GitHub with `[skip ci]`, so it doesn't start another build.
6. ArgoCD sees the changed manifest and syncs it to EKS, which does a rolling update of the app pods.
7. The AWS Load Balancer Controller keeps an internet-facing ALB pointed at the app pods. MySQL runs as a StatefulSet on EFS-backed storage.
8. Prometheus scrapes the cluster, nodes and Jenkins. Grafana shows dashboards, and Alertmanager emails you when something breaks.

### 0.2 What runs where

| Component                                                       | Runs on         | Created by                          |
|-----------------------------------------------------------------|-----------------|-------------------------------------|
| VPC, 3 public subnets, Internet Gateway                         | AWS             | Terraform <code>vpc/</code>         |
| Jenkins server (t3.small, Ubuntu 24.04)                         | EC2             | Terraform <code>ec2/</code>         |
| Prometheus + Grafana server (t3.micro)                          | EC2             | Terraform <code>ec2/</code>         |
| EKS cluster <code>testing_k8s</code> + 2 x c7i-flex.large nodes | EKS             | Terraform <code>eks/</code>         |
| Pod Identity roles for VPC CNI, EFS CSI, LB Controller          | IAM             | Terraform <code>eks/</code>         |
| EFS file system (MySQL data)                                    | EFS             | Terraform <code>efs/</code>         |
| ECR repository <code>python_web_application</code>              | ECR             | Terraform <code>ecr/</code>         |
| Route 53 zone + ACM certificate (optional)                      | Route 53 / ACM  | Terraform <code>Route53_ACM/</code> |
| EFS CSI driver, AWS Load Balancer Controller                    | EKS kube-system | Helm (Phase 2)                      |
| ArgoCD                                                          | EKS argocd      | kubectl (Phase 5)                   |
| Django app (Deployment) + MySQL (StatefulSet)                   | EKS k8s-project | ArgoCD                              |
| kube-prometheus-stack (Prometheus, Grafana, Alertmanager)       | EKS monitoring  | Helm (Phase 7)                      |

### 0.3 Repository layout

| Path                                    | Purpose                                                                                                                                                    |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Terraform/</code>                 | All AWS infrastructure (modules: <code>vpc</code> , <code>ec2</code> , <code>eks</code> , <code>efs</code> , <code>ecr</code> , <code>Route53_ACM</code> ) |
| <code>Jenkins_cicd/</code>              | Django app source, <code>Dockerfile</code> , <code>requirements.txt</code> , <code>Jenkinsfile</code>                                                      |
| <code>Kubernetes with ArgoCD/</code>    | Manifests that ArgoCD deploys (app, DB, ConfigMap, Secret, Ingress, StorageClass, RBAC)                                                                    |
| <code>Install and Configuration/</code> | Install guides, <code>install-jenkins-server.sh</code> , <code>argocd-application.yaml</code> , Prometheus/Alertmanager configs, Helm values               |
| <code>Security/</code>                  | Trivy config and scan script, NetworkPolicies, kube-bench job, security guide                                                                              |
| <code>docs/</code>                      | This runbook (Markdown + PDF) and the PDF generator                                                                                                        |

## 0.4 Time and cost

A full build-out takes about 2 to 3 hours the first time. Terraform alone takes about 20 minutes (EKS is the slow part).

**Cost warning:** The EKS control plane is billed per hour (about USD 0.10/hour) and is not covered by the free tier. You also pay for the ALBs, the EFS storage and any EC2 usage beyond free-tier hours. Run **Phase 11 (Teardown)** when you're done for the day. Everything can be recreated from this runbook.

# 1. Prerequisites

## 1.1 Tools on your workstation (Linux / WSL2 Ubuntu)

| Tool      | Minimum version                            | Check                                 |
|-----------|--------------------------------------------|---------------------------------------|
| Git       | 2.30                                       | <code>git --version</code>            |
| AWS CLI   | v2                                         | <code>aws --version</code>            |
| Terraform | 1.10 (needs S3 native locking)             | <code>terraform version</code>        |
| kubectl   | within one minor version of EKS            | <code>kubectl version --client</code> |
| Helm      | 3.x                                        | <code>helm version</code>             |
| Docker    | 24+ (optional, for local builds and scans) | <code>docker version</code>           |

Install the ones you are missing:

```
# AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install

# Terraform (HashiCorp apt repo)
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp.gpg
HC=https://apt.releases.hashicorp.com
echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg] $HC $(lsb_release -cs) main" \
| sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt-get update && sudo apt-get install -y terraform

# kubectl (latest stable)
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl

# Helm
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

## 1.2 AWS account and credentials

1. Sign in to the AWS console as an IAM user (not root). The user needs AdministratorAccess for this project, because Terraform creates IAM roles, a VPC, EKS and more.
2. Create an access key for the user: IAM > Users > your user > Security credentials > Create access key > Command Line Interface.
3. Configure the CLI:

```
aws configure
# AWS Access Key ID:      <your key>
# AWS Secret Access Key: <your secret>
# Default region name:    ap-south-1
# Default output format: json

aws sts get-caller-identity    # must print your Account and Arn
```

4. Create (or confirm) the EC2 key pair used for SSH. Terraform expects one named `vishwesh-EC2` (change it with `key_name` in `terraform.tfvars`):

```
aws ec2 describe-key-pairs --region ap-south-1 --query 'KeyPairs[].KeyName'
# if it is missing:
aws ec2 create-key-pair --region ap-south-1 --key-name vishwesh-EC2 \
  --query KeyMaterial --output text > ~/.ssh/vishwesh-EC2.pem
chmod 400 ~/.ssh/vishwesh-EC2.pem
```

### 1.3 GitHub repository and token

1. Use your fork: <https://github.com/gh-vishwesh/End-to-End-DevOps-Project>. App code and Kubernetes manifests live in this one repo. The Jenkinsfile, `argocd-application.yaml` and this runbook already point to it.
2. Create a fine-grained personal access token for Jenkins: GitHub > Settings > Developer settings > Personal access tokens > Fine-grained tokens > Generate new token.
3. Token settings: Repository access = *Only select repositories* > `End-to-End-DevOps-Project`. Permissions > Repository permissions > **Contents: Read and write** (Metadata: Read-only is added automatically). Set an expiry date.
4. Copy the token (starts with `github_pat_`). You'll paste it into Jenkins in Phase 3. Never commit it.

### 1.4 Clone the repo

```
git clone https://github.com/gh-vishwesh/End-to-End-DevOps-Project.git
cd End-to-End-DevOps-Project
```

## 2. Phase 1: Provision AWS infrastructure with Terraform

### Step 1.1: Create the Terraform state bucket (one time only)

Terraform keeps its state in S3 (`Terraform/provider.tf`: bucket `terraform-devops-backendfile`, key `terraform.tfstate`, `use_lockfile = true` for native S3 locking). The bucket must exist before `terraform init`.

```
# Bucket names are globally unique; if this one is taken, pick another and edit provider.tf
BUCKET=terraform-devops-backendfile
aws s3api create-bucket --bucket $BUCKET --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1
aws s3api put-bucket-versioning --bucket $BUCKET --versioning-configuration Status=Enabled
aws s3api put-bucket-encryption --bucket $BUCKET \
  --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}}'
aws s3api put-public-access-block --bucket $BUCKET \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Versioning lets you recover an older state file if one ever gets corrupted.

### Step 1.2: Set your variables

```
cd Terraform
cp example.tfvars terraform.tfvars      # terraform.tfvars is gitignored
curl -s ifconfig.me; echo                # your public IP
```

Edit `terraform.tfvars`:

```
admin_cidr = "<your-public-ip>/32"  # SSH + Prometheus/Grafana/Alertmanager UIs only from you
key_name   = "vishwesh-EC2"
domain_name = ""                    # e.g. "app.example.com", only if you own it (Phase 9)
```

If you leave `admin_cidr` at its default `0.0.0.0/0`, SSH and the monitoring UIs are open to the whole internet. Jenkins port 8080 is always public, because GitHub's webhooks have to reach it.

### Step 1.3: Init, plan, apply

```
terraform init          # downloads the AWS provider, connects to the S3 backend
terraform fmt -check    # formatting check (no output = OK)
terraform validate      # "Success! The configuration is valid."
terraform plan -out=tfplan
```

Read the plan. On a fresh account you should see **about 50 resources to add, 0 to change, 0 to destroy**. Then:

```
terraform apply tfplan # takes about 15-20 minutes; EKS cluster + node group are the slow part
```

### Step 1.4: Record the outputs

```
terraform output
```

| Output                                                 | Used in                                                        |
|--------------------------------------------------------|----------------------------------------------------------------|
| <code>eks_cluster_name</code>                          | <code>aws eks update-kubeconfig</code> (Phase 2)               |
| <code>vpc_id</code>                                    | AWS Load Balancer Controller install (Phase 2)                 |
| <code>efs_id</code>                                    | <code>Kubernetes with ArgoCD/storageclass.yml</code> (Phase 2) |
| <code>ecr_repository_url</code>                        | Check pushed images (Phase 3)                                  |
| <code>jenkins_public_ip, monitoring_public_ip</code>   | SSH to the servers, Jenkins URL, GitHub webhook                |
| <code>server_private_ips</code>                        | Prometheus scrape targets (Phase 8)                            |
| <code>acm_certificate_arn, route53_name_servers</code> | HTTPS (Phase 9, optional)                                      |

### Step 1.5: Verify in AWS

```
aws eks describe-cluster --name testing_k8s --region ap-south-1 --query cluster.status      # "ACTIVE"
aws eks list-nodegroups --cluster-name testing_k8s --region ap-south-1
aws ecr describe-repositories --region ap-south-1 --query 'repositories[].repositoryUri'
aws efs describe-file-systems --region ap-south-1 --query 'FileSystems[][FileSystemId,LifecycleState]'
```

What Terraform created:

- **VPC** 10.0.0.0/16 with 3 public subnets in 3 AZs, tagged `kubernetes.io/role/elb=1` so the ALB controller can find them.
- **EC2 servers** (Jenkins, Prometheus and Grafana): latest Ubuntu 24.04 AMI, 20 GB encrypted gp3 disk, IMDSv2 only, and an instance profile with ECR push rights (so no AWS keys are stored on Jenkins).
- **EKS**: cluster `testing_k8s`, managed node group (2 x c7i-flex.large, AL2023), and add-ons vpc-cni (with network policy enabled), kube-proxy, coreDNS and eks-pod-identity-agent.
- **IAM**: least-privilege node role. The EFS CSI driver and AWS Load Balancer Controller get Pod Identity roles.
- **EFS**: encrypted, mount target in each subnet, NFS (2049) allowed only from the EKS security groups.
- **ECR**: `python_web_application` with scan-on-push and a lifecycle policy that keeps 20 images.

## 3. Phase 2: Connect to EKS and install cluster add-ons

### Step 2.1: Configure kubectl

```
aws eks update-kubeconfig --region ap-south-1 --name testing_k8s
kubectl config current-context      # arn:aws:eks:ap-south-1:<account>:cluster/testing_k8s
kubectl get nodes -o wide           # 2 nodes, STATUS Ready
kubectl get pods -n kube-system     # aws-node, coredns, kube-proxy, eks-pod-identity-agent Running
```

If you work with several clusters, list them with `kubectl config get-contexts` and switch with `kubectl config use-context <name>`.

### Step 2.2: Install the EFS CSI driver

The driver lets Kubernetes create volumes on EFS dynamically. Terraform already linked the service account `efs-csi-controller-sa` to an IAM role through Pod Identity, so keep that exact name.

```
helm repo add aws-efs-csi-driver https://kubernetes-sigs.github.io/aws-efs-csi-driver/
helm repo update
helm upgrade -i aws-efs-csi-driver aws-efs-csi-driver/aws-efs-csi-driver -n kube-system \
  --set controller.serviceAccount.create=true \
  --set controller.serviceAccount.name=efs-csi-controller-sa
kubectl get pods -n kube-system -l app.kubernetes.io/name=aws-efs-csi-driver # controller + node pods
```

### Step 2.3: Install the AWS Load Balancer Controller

This controller turns the Kubernetes Ingress into an AWS Application Load Balancer.

```
VPC_ID=$(cd Terraform && terraform output -raw vpc_id)
helm repo add eks https://aws.github.io/eks-charts
helm repo update
helm upgrade -i aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=testing_k8s \
  --set region=ap-south-1 \
  --set vpcId="$VPC_ID" \
  --set serviceAccount.create=true \
  --set serviceAccount.name=aws-load-balancer-controller
kubectl get deploy -n kube-system aws-load-balancer-controller # READY 2/2
kubectl logs -n kube-system deploy/aws-load-balancer-controller | tail # no AccessDenied errors
```

### Step 2.4: Point the StorageClass at the new EFS file system

The EFS ID changes every time you recreate the infrastructure.

```
EFS_ID=$(cd Terraform && terraform output -raw efs_id)
sed -i "s/fileSystemId: ./fileSystemId: $EFS_ID/" "Kubernetes with ArgoCD/storageclass.yml"
grep fileSystemId "Kubernetes with ArgoCD/storageclass.yml"
git add "Kubernetes with ArgoCD/storageclass.yml"
git commit -m "Point StorageClass at EFS $EFS_ID"
git push
```

## 4. Phase 3: Jenkins CI server

### Step 3.1: SSH to the Jenkins server

```
JENKINS_IP=$(cd Terraform && terraform output -raw jenkins_public_ip)
echo $JENKINS_IP
ssh -i ~/.ssh/vishwesh-EC2.pem ubuntu@$JENKINS_IP
```

If SSH times out, your public IP has probably changed since you set `admin_cidr`. Update `terraform.tfvars` and run `terraform apply` again.

### Step 3.2: Install Docker, AWS CLI, Java 21 and Jenkins

From your workstation, copy the installer and run it:

```
scp -i ~/.ssh/vishwesh-EC2.pem "Install and Configuration/install-jenkins-server.sh" ubuntu@$JENKINS_IP:~
ssh -i ~/.ssh/vishwesh-EC2.pem ubuntu@$JENKINS_IP 'sudo bash install-jenkins-server.sh'
```

The script adds 2 GB of swap, installs Docker Engine from Docker's repo, AWS CLI v2, OpenJDK 21 and Jenkins LTS (2026 signing key), adds `jenkins` to the `docker` group, and finally prints the initial admin password. The manual equivalent is in `Install and Configuration/Jenkins, Docker and AWS Cli installation.txt`.

Verify on the server:

```
sudo -u jenkins docker ps          # no "permission denied"
sudo -u jenkins aws sts get-caller-identity # assumed-role/Terraform_jenkins_ec2roleforecr/...
systemctl is-active jenkins        # active
```

### Step 3.3: Unlock Jenkins

1. Open `http://<JENKINS_IP>:8080`.
2. Paste the initial admin password (`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`).
3. Choose **Install suggested plugins**. These include Pipeline, Git, GitHub and Credentials Binding, which the Jenkinsfile needs.
4. Create your admin user, and keep `http://<JENKINS_IP>:8080/` as the Jenkins URL.
5. Optional, for monitoring: Manage Jenkins > Plugins > Available > install **Prometheus metrics**. Metrics then appear at `/prometheus`.

### Step 3.4: Add the GitHub token as a credential

Manage Jenkins > Credentials > System > Global credentials (unrestricted) > **Add Credentials**:

| Field       | Value                                                          |
|-------------|----------------------------------------------------------------|
| Kind        | Secret text                                                    |
| Scope       | Global                                                         |
| Secret      | your <code>github_pat_...</code> token from step 1.3           |
| ID          | <code>GITHUB_TOKEN</code> (must match the Jenkinsfile exactly) |
| Description | GitHub PAT for End-to-End-DevOps-Project                       |

### Step 3.5: Create the pipeline job

1. Dashboard > **New Item** > name `End-to-End-DevOps-Project` > **Pipeline** > OK.
2. Under *General*, tick **GitHub project** and set Project url  
`https://github.com/gh-vishwesh/End-to-End-DevOps-Project/`.
3. Under *Triggers*, tick **GitHub hook trigger for GITScm polling**.

4. Under *Pipeline*, set Definition to **Pipeline script from SCM**, then:

| Field            | Value                                                                                     |
|------------------|-------------------------------------------------------------------------------------------|
| SCM              | Git                                                                                       |
| Repository URL   | <code>https://github.com/gh-vishwesh/End-to-End-DevOps-Project.git</code>                 |
| Credentials      | none (the repo is public; the pipeline uses <code>GITHUB_TOKEN</code> itself for pushing) |
| Branch Specifier | <code>*/main</code>                                                                       |
| Script Path      | <code>Jenkins_cicd/Jenkinsfile</code>                                                     |

5. Click **Save**.

### Step 3.6: Add the GitHub webhook

GitHub > your repo > Settings > Webhooks > **Add webhook**:

| Field        | Value                                                                                 |
|--------------|---------------------------------------------------------------------------------------|
| Payload URL  | <code>http://&lt;JENKINS_IP&gt;:8080/github-webhook/</code> (keep the trailing slash) |
| Content type | application/json                                                                      |
| Events       | Just the push event                                                                   |
| Active       | ticked                                                                                |

After you save, GitHub sends a ping. Under *Recent Deliveries* it should show a green tick (HTTP 200).

The Jenkins public IP changes if the instance is stopped and started. If it does, update the webhook URL. For a fixed IP, attach an Elastic IP.

### Step 3.7: Run the first build

Click **Build Now** once by hand. This also registers the pipeline's parameters and webhook trigger. These are the stages and what each one does:

| Stage                    | What happens                                                                                                                                                                           | Fails when                                               |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Clone Repo               | Fresh clone of <code>main</code> ; skips the build if the last commit has <code>[skip ci]</code> ; works out the AWS account, ECR URL and tag <code>build-&lt;N&gt;-&lt;sha&gt;</code> | Bad token, repo URL wrong                                |
| Unit Tests               | <code>python manage.py test</code> in a <code>python:3.12-slim</code> container using SQLite                                                                                           | Any Django test fails                                    |
| Security Scan: Repo      | Trivy secret scan (blocking) + IaC/dependency report                                                                                                                                   | A secret (token, key) is committed                       |
| Build Docker Image       | Multi-stage build: Python 3.12 slim, gunicorn, non-root UID 10001                                                                                                                      | Dockerfile/dependency error                              |
| Security Scan: Image     | Trivy image report + gate                                                                                                                                                              | Fixable CRITICAL CVE (if <code>FAIL_ON_CRITICAL</code> ) |
| Push Image To ECR        | <code>docker login</code> with the instance role, then push                                                                                                                            | Instance role lacks ECR rights                           |
| Update ArgoCD Manifest   | <code>sed</code> swaps the <code>image:</code> line in <code>appdeployment.yml</code>                                                                                                  | Manifest path changed                                    |
| Commit and Push Manifest | Commit <code>Update app image to build-N-sha [skip ci]</code> , rebase, push                                                                                                           | Token lacks Contents: write                              |

Verify:

```
aws ecr list-images --repository-name python_web_application --region ap-south-1
git pull && grep image: "Kubernetes with ArgoCD/appdeployment.yml" # now points at your ECR image
```

Then open the build page, click **Artifacts**, and check the Trivy reports (`trivy-repo.txt`, `trivy-image.txt`).

Run this first build **before** you create the ArgoCD application. Until Jenkins has pushed an image, `appdeployment.yml` points at the placeholder tag `:initial`, which doesn't exist in ECR. The pods would sit in `ImagePullBackOff` until the first build replaces it.

## 5. Phase 4: Security scanning (DevSecOps)

All security tooling lives in `Security/` and is described in `Security/README.md`. In short:

1. **In the pipeline (automatic):** Trivy secret scan (blocking), IaC/dependency report, image scan (blocking on fixable CRITICAL), and ECR scan-on-push.
2. **Before you push (local):**

```
./Security/scripts/run-security-scans.sh # repo
docker build -t python_web_application:local Jenkins_cicd
./Security/scripts/run-security-scans.sh python_web_application:local # repo + image
ls reports/ # secrets.txt misconfig.txt dependencies.txt image.txt
```

3. **In the cluster (after Phase 5):** NetworkPolicies (`Security/kubernetes/network-policies.yml`), the CIS EKS benchmark (`Security/kubernetes/kube-bench-job.yml`) and Pod Security Standards labels. The step-by-step commands are in `Security/README.md`, sections 3 to 5.

## 6. Phase 5: GitOps with ArgoCD

### Step 5.1: Install ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd -w # wait until every pod is Running, then Ctrl+C
```

### Step 5.2: Open the UI and log in

```
kubectl port-forward -n argocd svc/argocd-server 8080:443
# open https://localhost:8080 and accept the self-signed certificate

# password for user "admin" (second terminal):
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo
```

Change the password under User Info > Update Password, then delete the bootstrap secret: `kubectl -n argocd delete secret argocd-initial-admin-secret`.

To expose ArgoCD publicly instead of port-forwarding: `kubectl patch svc argocd-server -n argocd -p '{"spec":{"type":"LoadBalancer"}}'`, then use the EXTERNAL-IP from `kubectl get svc argocd-server -n argocd`.

### Step 5.3: Create the Application

```
kubectl apply -f "Install and Configuration/argocd-application.yaml"
kubectl get applications -n argocd # myapp Synced Healthy (after 1-3 minutes)
```

This Application tracks `https://github.com/gh-vishwesh/End-to-End-DevOps-Project.git`, branch `main`, path `Kubernetes with ArgoCD`, and deploys into the namespace `k8s-project` (created automatically). Automated sync is on, with **prune** (resources deleted from Git are deleted from the cluster) and **self-heal** (manual `kubectl edit` changes are reverted to match Git).

### Step 5.4: Check what ArgoCD deployed

```
kubectl get all,pvc,ingress,configmap,secret -n k8s-project
```

| Resource                         | Expected state                                                               |
|----------------------------------|------------------------------------------------------------------------------|
| statefulset/db-pod, pod db-pod-0 | Running 1/1 (first start takes 1-2 min while MySQL initialises on EFS)       |
| pvc/mysql-storage-db-pod-0       | Bound, StorageClass <code>efs-sc</code>                                      |
| deployment/app-pod               | 2/2 Ready                                                                    |
| service/myapp, service/mydb      | ClusterIP                                                                    |
| ingress/myapp-ingress            | ADDRESS = <code>myapp-alb-xxxx.ap-south-1.elb.amazonaws.com</code> (2-4 min) |

### Step 5.5: Open the application

```
APP_URL=$(kubectl get ingress myapp-ingress -n k8s-project \
-o jsonpath='{.status.loadBalancer.ingress[0].hostname}')
echo "http://$APP_URL/"
curl -I "http://$APP_URL/" # HTTP/1.1 200 OK
```

In the browser, insert a record, then open `/showpage` to see it listed, and try **Edit** and **Delete**. The data lives in MySQL on EFS, so it survives pod restarts. To prove it, run `kubectl delete pod db-pod-0 -n k8s-project`; the record is still there once the pod comes back.

## 7. Phase 6: Prove the full CI/CD loop

1. Make a visible change, for example the heading in `Jenkins_cicd/app/templates/show.html`:

```
sed -i 's|<h2>Register Data</h2>|<h2>Register Data - v2</h2>|' Jenkins_cicd/app/templates/show.html
git commit -am "Change page heading" && git push
```

2. **GitHub**: the webhook delivery shows 200.
3. **Jenkins**: a new build starts within seconds and every stage turns green.
4. **GitHub**: a new commit by *Jenkins CI*, "Update app image to build-N-sha [skip ci]", appears. Jenkins also starts a build for it, which stops at *Clone Repo* with status NOT\_BUILT. That is the loop protection working, not an error.
5. **ArgoCD**: within about 3 minutes (the default polling interval; click **Refresh** to speed it up) the app goes OutOfSync, then Syncing, then Synced, with a rolling update.
6. Watch the rollout: `kubectl rollout status deploy/app-pod -n k8s-project`.
7. Reload `http://$APP_URL/showpage`: the heading now reads "Register Data - v2".

To **roll back**, revert the Jenkins manifest commit (`git revert <sha> && git push`), or in ArgoCD open History and Rollback. Note that a manual rollback in ArgoCD is overridden by the next auto-sync, so reverting in Git is the GitOps way.

## 8. Phase 7: In-cluster monitoring (kube-prometheus-stack)

This is the recommended way to monitor EKS. It installs Prometheus, Alertmanager, Grafana, kube-state-metrics and node-exporter in one Helm release.

### Step 7.1: Install

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm upgrade -i monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace \
  -f "Install and Configuration/kube-prometheus-stack-values.yaml" \
  --set grafana.adminPassword='<choose-a-strong-password>'
kubectl get pods -n monitoring          # all Running after 2-3 minutes
```

What the values file does: it turns off scraping for control-plane parts that EKS hides (scheduler, controller-manager, etcd, kube-proxy), sets Prometheus retention to 3 days with resource limits, gives Grafana enough memory (768Mi), and puts Grafana behind its own internet-facing ALB ([grafana-alb](#)).

### Step 7.2: Open Grafana

```
kubectl get ingress -n monitoring          # ADDRESS = grafana-alb-xxxx.ap-south-1.elb.amazonaws.com
```

Log in as [admin](#) with the password you set. Useful built-in dashboards (Dashboards > Browse):

- *Kubernetes / Compute Resources / Namespace (Pods)*: choose namespace [k8s-project](#) to see CPU and memory per app/DB pod.
- *Kubernetes / Compute Resources / Node (Pods)*: node usage.
- *Node Exporter / Nodes*: OS-level metrics of each EKS worker.
- *Alertmanager / Overview*: alerts that are firing.

### Step 7.3: Open Prometheus and Alertmanager (port-forward)

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090 # http://localhost:9090
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-alertmanager 9093 # http://localhost:9093
```

In Prometheus, Status > Targets should be all UP. Try these queries: [up](#),  
[kube\\_pod\\_status\\_ready{namespace="k8s-project"}](#),  
[container\\_memory\\_working\\_set\\_bytes{namespace="k8s-project"}](#).

### Step 7.4: Email alerts and Jenkins metrics (optional)

[kube-prometheus-stack-values.yaml](#) has commented examples for both:

1. **Email**: fill in [alertmanager.config](#) with your Gmail address and a Gmail App Password. Keep it in a separate, uncommitted values file and pass it with [-f](#).
2. **Jenkins**: uncomment [additionalScrapeConfigs](#) and set [<JENKINS\\_PRIVATE\\_IP>](#) (from [terraform output server\\_private\\_ips](#)). This needs the Jenkins Prometheus metrics plugin.

Re-run the [helm upgrade](#) command from step 7.1 to apply the changes.

## 9. Phase 8: Monitoring server on EC2 (Prometheus, Alertmanager, Grafana)

The second EC2 ("Prometheus and Grafana") gives you a classic, VM-based monitoring setup. It watches the Jenkins server, the pipeline, and itself, and sends email alerts. The full commands are in [Install and Configuration/](#), and the order is:

### Step 8.1: Prometheus

```
MON_IP=$(cd Terraform && terraform output -raw monitoring_public_ip)
ssh -i ~/.ssh/vishwesh-EC2.pem ubuntu@$MON_IP
```

Follow [Prometheus installation.txt](#), part 1: download v3.3.0, create the `prometheus` user, create `/etc/prometheus/rules` and `/data`, write the systemd unit, then enable and start it. Check with `curl -s localhost:9090/-/ready`.

### Step 8.2: node\_exporter on both servers

On the monitoring server **and** on the Jenkins server, follow [Prometheus installation.txt](#), part 2 (node\_exporter v1.9.1 on port 9100, with `--collector.systemd` so the SSH-service alert works). The security group allows all traffic inside the VPC, so Prometheus can reach Jenkins on port 9100.

### Step 8.3: Pushgateway and Alertmanager

Follow parts 3 and 4 of [Prometheus installation.txt](#). Pushgateway listens on 9091 and Alertmanager on 9093.

### Step 8.4: Configuration and alert rules

1. Copy the content of `prometheus.yml file.txt` into `/etc/prometheus/prometheus.yml`.
2. Replace `<JENKINS_PRIVATE_IP>`, `<JENKINS_USER>` and `<JENKINS_API_TOKEN>` (Jenkins > your user > Security > API Token > Add new token).
3. Delete the four Kubernetes jobs unless you also want this server to scrape EKS; the in-cluster stack from Phase 7 already does that. To keep them, follow [Kubernetes monitoring configuration.txt](#).
4. Copy `Alert Rules.txt` into `/etc/prometheus/rules/allrules.yml`. It defines 8 rules: InstanceDown, HighCPUUsage, HighMemoryUsage, DiskFull, RootDiskFull, SSHDDown, JenkinsJobFailed and JenkinsJobSucceeded.
5. Validate and reload:

```
sudo chown -R prometheus:prometheus /etc/prometheus
promtool check config /etc/prometheus/prometheus.yml      # SUCCESS ... 8 rules found
curl -X POST localhost:9090/-/reload
```

In `http://<MON_IP>:9090`, Status > Targets should show prometheus, node\_exporter (x2), pushgateway and jenkins\_job all UP.

### Step 8.5: Email notifications

Follow [Email Alert.txt](#). Turn on 2-Step Verification on your Google account, create an App Password, put it into `/etc/alertmanager/alertmanager.yml`, run `amtool check-config`, then restart Alertmanager. To test, stop node\_exporter on Jenkins for 2 minutes; an *InstanceDown* email should arrive, followed by a *resolved* email once you start it again.

### Step 8.6: Grafana

Follow [Grafana installation.txt](#). Grafana runs on port 3000; the first login is admin/admin and you must change the password. Then:

1. Connections > Data sources > Add > Prometheus > URL `http://localhost:9090` > Save & test.

2. Dashboards > New > Import > ID **1860** (Node Exporter Full) > choose the Prometheus data source.
3. Optional: import ID **9964** (Jenkins: Performance and Health Overview) for pipeline metrics.

## 10. Phase 9: HTTPS with Route 53 and ACM (optional)

You need a domain you control.

1. Set `domain_name = "app.example.com"` in `terraform.tfvars`, then run `terraform apply`.
2. Copy `terraform output route53_name_servers` into your registrar's name-server settings. ACM validation waits (up to 30 minutes) until DNS has propagated.
3. Edit `Kubernetes with ArgoCD/ingress.yml`: set `listen-ports` to `'[{"HTTP": 80}, {"HTTPS": 443}]'`, then uncomment `certificate-arn` (value from `terraform output -raw acm_certificate_arn`) and `ssl-redirect`.
4. Add an `env` entry `DJANGO_CSRF_TRUSTED_ORIGINS=https://app.example.com` to `appdeployment.yml`. Django 4+ rejects HTTPS form posts from origins it doesn't trust.
5. Commit and push. ArgoCD updates the ALB.
6. Create a Route 53 alias record from `app.example.com` to the ALB hostname (console: Hosted zone > Create record > Alias > Application Load Balancer > ap-south-1 > myapp-alb).

## 11. Verification checklist

| #  | Check                 | Command / place                                                           | Expected                                                 |
|----|-----------------------|---------------------------------------------------------------------------|----------------------------------------------------------|
| 1  | Terraform state in S3 | <code>aws s3 ls s3://terraform-devops-backendfile</code>                  | <code>terraform.tfstate</code>                           |
| 2  | EKS nodes ready       | <code>kubectl get nodes</code>                                            | 2 x Ready                                                |
| 3  | Controllers           | <code>kubectl get deploy -n kube-system</code>                            | aws-load-balancer-controller 2/2, efs-csi-controller 2/2 |
| 4  | Jenkins build         | Jenkins UI                                                                | All stages green, Trivy reports archived                 |
| 5  | Image in ECR          | <code>aws ecr list-images --repository-name python_web_application</code> | <code>build-N-sha</code> tags                            |
| 6  | Manifest bumped       | GitHub commits                                                            | "Update app image ... [skip ci]" by Jenkins CI           |
| 7  | ArgoCD                | <code>kubectl get applications -n argocd</code>                           | Synced / Healthy                                         |
| 8  | App pods              | <code>kubectl get pods -n k8s-project</code>                              | app-pod x2 + db-pod-0 Running                            |
| 9  | Storage               | <code>kubectl get pvc -n k8s-project</code>                               | Bound (efs-sc)                                           |
| 10 | App via ALB           | <code>curl -I http://&lt;alb&gt;/</code>                                  | 200 OK                                                   |
| 11 | Grafana               | <code>http://&lt;grafana-alb&gt;/</code>                                  | Dashboards show k8s-project pods                         |
| 12 | Alerts                | Stop <code>node_exporter</code> on Jenkins                                | Email received                                           |
| 13 | Security              | <code>./Security/scripts/run-security-scans.sh</code>                     | "Security scans passed."                                 |

## 12. Troubleshooting

### Terraform

- **\*\*Error: Failed to get existing workspaces: S3 bucket does not exist:\*\*** create the bucket (Step 1.1), or fix the bucket name in `provider.tf`.
- **\*\*InvalidKeyPair.NotFound:\*\*** the key pair named in `key_name` doesn't exist in ap-south-1 (section 1.2, item 4).
- **\*\*VcpuLimitExceeded / InsufficientInstanceCapacity:\*\*** request a higher On-Demand vCPU quota (Service Quotas > EC2 > Running On-Demand Standard instances), or reduce `desired_size` in `eks/eks.tf`.
- **State lock error:** someone else, or a crashed run, holds the lock. Once you are sure no apply is running: `terraform force-unlock <LOCK_ID>`.
- **\*\*terraform destroy hangs on subnets/VPC:\*\*** ALBs and ENIs created by Kubernetes still exist. Delete the Ingresses first (Phase 11 order).

### Jenkins

- **\*\*permission denied ... /var/run/docker.sock:\*\*** run `sudo usermod -aG docker jenkins && sudo systemctl restart jenkins`.
- **Clone fails with 403/404:** the `GITHUB_TOKEN` credential is missing or wrong, the token has expired, or it doesn't include this repository.
- **\*\*Push fails with Permission ... denied / 403: the token needs Contents: Read and write\*\*.**
- **Webhook does not trigger:** check *Recent Deliveries* in GitHub. The URL must end in `/github-webhook/`, port 8080 must be open (it is in the security group), "GitHub hook trigger" must be ticked, and at least one manual build must have run.
- **Build loops forever:** the Jenkins commit must contain `[skip ci]`; the Clone Repo stage checks for it.
- **Security Scan: Image fails:** open `trivy-image.txt` in the build artifacts. Usually a newer base image fixes it, so just rebuild (`docker build --pull`). If you must ship anyway, run *Build with Parameters* with `FAIL_ON_CRITICAL` unticked, or add the CVE with a reason to `Security/trivy/.trivyignore`.
- **\*\*ECR push no basic auth credentials / AccessDenied:\*\*** run `aws sts get-caller-identity` on the server. It must show the `Terraform_jenkins_ec2roleforecr` role.
- **Jenkins slow or killed:** check `free -m`. The installer adds 2 GB swap; you can also raise the Jenkins instance type in `ec2/variables.tf`.

### Kubernetes / ArgoCD

- **\*\*App pods ImagePullBackOff:\*\*** the image tag doesn't exist yet (run the Jenkins build), or the node role lacks ECR read (`AmazonEC2ContainerRegistryReadOnly` is attached by Terraform).
- **\*\*db-pod-0 Pending, PVC Pending:\*\*** the `filesystemId` in `storageclass.yml` is wrong (Step 2.4), or the EFS CSI driver isn't running. Look at `kubectl describe pvc -n k8s-project`.
- **\*\*App CrashLoopBackOff / 500 errors:\*\*** the DB isn't ready yet, or the password doesn't match. Check `kubectl logs deploy/app-pod -n k8s-project` and `kubectl logs db-pod-0 -n k8s-project`. MySQL only applies `MYSQL_ROOT_PASSWORD` on an empty volume.
- **Ingress has no ADDRESS:** check `kubectl logs -n kube-system deploy/aws-load-balancer-controller` and `kubectl describe ingress myapp-ingress -n k8s-project`. Subnets must carry `kubernetes.io/role/elb=1` (set by Terraform).
- **ALB returns 502/503:** the targets are unhealthy. The readiness probe / must return 200: `kubectl get endpoints myapp -n k8s-project`.

- **ArgoCD OutOfSync but not syncing:** click Refresh, then check the app's events. ArgoCD shows YAML errors in the UI.
- **Grafana OOMKilled:** raise `grafana.resources.limits.memory` in the values file, then run `helm upgrade` again.

### Useful commands

```
kubectl get events -n k8s-project --sort-by=.lastTimestamp | tail -20
kubectl describe pod <pod> -n k8s-project
kubectl logs -f deploy/app-pod -n k8s-project
kubectl exec -it db-pod-0 -n k8s-project -- mysql -uroot -p django_crud -e "select * from app_register;"
kubectl top pods -n k8s-project # needs metrics-server
sudo journalctl -u jenkins -f # on the Jenkins server
aws eks describe-cluster --name testing_k8s --region ap-south-1 --query cluster.status
```

## 13. Phase 10: Day-2 operations

- **Scale the app:** change `replicas` in `appdeployment.yml`, then commit and push (GitOps, not `kubectl scale`, because self-heal would revert it).
- **Change configuration:** edit `configmap.yml` or `secret.yml`, then commit. Restart the pods to pick up env changes: `kubectl rollout restart deploy/app-pod -n k8s-project`.
- **Upgrade EKS:** bump the cluster version in Terraform (set `version` on `aws_eks_cluster`), apply, then update the node group and add-ons.
- **Rotate the GitHub token:** create a new PAT, then edit the `GITHUB_TOKEN` credential in Jenkins.
- **Back up MySQL:** `kubectl exec db-pod-0 -n k8s-project -- mysqldump -uroot -p<pw> django_crud > backup.sql`. EFS also supports AWS Backup.

## 14. Phase 11: Teardown (avoid charges)

Order matters: Kubernetes-created ALBs and their security groups must go before Terraform deletes the VPC.

```
# 1. Remove the app (deletes the app ALB) and the monitoring stack (deletes the Grafana ALB)
kubectl delete -f "Install and Configuration/argocd-application.yaml"
kubectl delete ingress --all -n k8s-project
helm uninstall monitoring -n monitoring
kubectl delete svc argocd-server -n argocd --ignore-not-found # only if you made it a LoadBalancer

# 2. Wait until no ALBs remain
aws elbv2 describe-load-balancers --region ap-south-1 --query 'LoadBalancers[].LoadBalancerName'

# 3. Remove the controllers, then the infrastructure
helm uninstall aws-load-balancer-controller -n kube-system
helm uninstall aws-efs-csi-driver -n kube-system
cd Terraform && terraform destroy # about 15 minutes
```

The S3 state bucket is kept, so you can redeploy later. To remove it too, empty it (all versions) and then delete it. Also remove the GitHub webhook, or it will report failed deliveries.

## 15. Security summary

| Area         | Control                                                                                                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Network      | Security groups with explicit ports; SSH/monitoring limited to <code>admin_cidr</code> ; EFS only from EKS; NetworkPolicies (default deny) |
| Identity     | Instance profile for Jenkins (no static keys); EKS Pod Identity for controllers; least-privilege node role                                 |
| Data         | Encrypted EFS, EBS, ECR, S3 state; IMDSv2 required                                                                                         |
| Supply chain | Pinned dependencies; Trivy secret, IaC, dependency and image scans in CI; ECR scan-on-push                                                 |
| Runtime      | Non-root container, read-only root filesystem, no privilege escalation, all capabilities dropped, seccomp RuntimeDefault                   |
| Compliance   | kube-bench CIS EKS benchmark; Pod Security Standards labels                                                                                |
| Known gap    | <code>secret.yml</code> is base64 only. Move to Sealed Secrets ( <a href="#">Security/README.md</a> , section 7)                           |